

操作系统第三次作业

姓名：陈天乐

班级：无25

学号：

Q1:什么是临界资源？什么是临界区？

临界资源(critical resource)指的是系统中某些共享资源一次只允许一个进程使用。而临界区(critical section)是值进程中访问临界资源的那部分代码片段。

Q2:什么是信号量？信号量s的计数值 $s.count > 0$ ， $s.count = 0$ 和 $s.count < 0$ 的意义分别是什么？

信号量是Dijkstra提出的概念，是一种同步对象，也是用于管理临界资源的一个机制。每个信号量s会包含一个整数值 $s.count$ ，以及一个进程等待队列 $s.queue$ ，这个队列中是阻塞在该信号量上的各个进程的标识。

- $s.count > 0$ 意味着目前有 $count$ 个资源是可用的
- $s.count = 0$ 表示现在无资源可用，即将进入等待
- $s.count < 0$ 表示有 $|count|$ 个进程正在 $s.queue$ 中等待资源

Q3:当低优先级进程正处在临界区之中时，如果高优先级进程变为就绪并被调度，若采用忙等则产生优先级反转问题。优先级反转问题是否也适用于线程？分别考虑ULT和KLT。

对于ULT而言，线程全部在用户态下实现，线程调度是不依靠硬件中断的，完全依靠用户空间的线程库管理，若线程库不涉及抢占式机制的调度，采用协作式调度的话，这个时候只有低优先级的线程让出CPU后，才会让高优先级线程使用CPU，因此此情况下优先级反转情况较少。

对于KLT而言，线程是由内核实现，每个线程拥有独立的优先级，内核基于这些优先级进行抢占式调度，那么这种情况下线程和进程的管理情况是一样的，会产生优先级反转问题。

Q4：假使A、B两个火车站之间是单线铁路，许多列车可以同时到达A站，然后经A站到B站，又列车从A到B的行驶时间是 t ，列车到B站后的停留时间是 $t/2$ 。在该问题模型中，哪些是临界资源？

首先，因为A、B两个站之间是单线铁路，但是A到B的时间是 t ，列车到达B站后停留的时间是 $t/2$ 。考虑第一辆车从A站出发，在行驶到A、B站中点时第二辆车就可以出发，这样在第二辆车即将到达B站时，第一辆车即将从B站离开。因此我们会发现，A、B之间只有A到A、B中点这段铁路是临界资源。

Q5:苹果-桔子问题：桌子上有一只盘子，每次只能向其中放入一个水果，要求：1.爸爸专向盘子里放苹果，女儿专等吃盘子中的苹果；2.妈妈专向盘子里放桔子，儿子专等吃盘子中的桔子；3.只有盘子为空时，爸爸或妈妈才可向盘子中放入一个水果；4.仅当盘子中有自己需要的水果时，儿子或女儿可以从盘子中取出。使用信号量和P、V操作使爸爸、妈妈、儿子和女儿正确同步工作。

该问题可以抽象成为生产者和消费者问题，那么解决该问题的思路是需使用信号量协调生产者（爸爸、妈妈）和消费者（女儿、儿子）的操作。以下是基于信号量（empty、apple、orange）的同步方案：

- empty：表示盘子是否为空，初始值为 1（初始时盘子为空）。
- apple：表示盘子中是否有苹果，初始值为 0（初始时无苹果）。
- orange：表示盘子中是否有桔子，初始值为 0（初始时无桔子）。
- 生产者必须等待盘子为空（P(empty)）才能放入水果。放入水果后，通知对应的消费者（V(apple) 或 V(orange)）。
- 消费者必须等待盘子中有对应的水果（P(apple) 或 P(orange)）才能取出。取出水果后，释放盘子为空状态（V(empty)）。

实现伪代码如下：

```
semaphore empty = 1; // 盘子空信号量
semaphore apple = 0; // 苹果信号量
semaphore orange = 0; // 桔子信号量

// 爸爸进程
void father() {
    while (true) {
        P(empty); // 等待盘子为空
        放入苹果;
        V(apple); // 通知女儿有苹果
    }
}

// 妈妈进程
void mother() {
    while (true) {
        P(empty); // 等待盘子为空
        放入桔子;
        V(orange); // 通知儿子有桔子
    }
}

// 女儿进程
void daughter() {
```

```
    while (true) {  
        P(apple);    // 等待苹果  
        取出苹果;  
        V(empty);    // 释放盘子为空  
    }  
}  
  
// 儿子进程  
void son() {  
    while (true) {  
        P(orange);    // 等待桔子  
        取出桔子;  
        V(empty);    // 释放盘子为空  
    }  
}
```